

Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Partea a II-a

Faza de proiectare

Apetrei Diana-Andreea

Grupa: 30231

Mai 2026

Cuprins

1	Faza de proiectare	2
1.1	Arhitectura și Principiile Domain-Driven Design (DDD)	2
1.2	Implementarea Componentei API Gateway	2
1.2.1	Tehnologii și Dependențe Utilizate	2
1.2.2	Configurarea Rutelor (Routing)	2
1.2.3	Avantajele aduse arhitecturii	3
1.3	Harta de context (Context Map)	4
1.4	Arhitectura internă a microserviciilor (Privire de ansamblu)	5
1.5	Arhitectura Aplicației Client (MVVM)	6
1.6	Diagrama de Pachete	6
1.7	Diagrame de Clase	6
1.7.1	Utilizator Neautentificat	7
1.7.2	Utilizator tip Angajat	8
1.7.3	Utilizator tip Manager	9
1.7.4	Utilizator tip Administrator	10
1.8	Diagrama de Componente	11
1.9	Diagrama de Dezvoltare (Deployment)	12

1 Faza de proiectare

În această secțiune sunt prezentate deciziile arhitecturale și de design ale sistemului software pentru agenția de rezervare a biletelor de avion. Arhitectura aleasă se bazează pe o separare clară a responsabilităților, folosind un stil arhitectural distribuit, orientat pe microservicii pentru partea de backend, și modelul MVVM (Model-View-ViewModel) pentru aplicația client.

1.1 Arhitectura și Principiile Domain-Driven Design (DDD)

Pentru dezvoltarea backend-ului, s-a optat pentru o abordare bazată pe principiile **Domain-Driven Design (DDD)**. Această metodologie permite descompunerea complexității sistemului în modele de domeniu izolate și coezive (Bounded Contexts), facilitând dezvoltarea, scalarea și mentenanța independentă a fiecărei componente a sistemului.

Principiul fundamental aplicat în proiectarea sistemului a fost **Separarea Preocupărilor (Separation of Concerns)** combinat cu **Principiul Inversiunii Dependențelor (Dependency Inversion Principle - DIP)**.

1.2 Implementarea Componentei API Gateway

Într-o arhitectură distribuită bazată pe microservicii, expunerea directă a fiecărui serviciu către aplicația client (JavaFX) reprezintă un anti-șablon (*anti-pattern*), deoarece crește gradul de cuplare, complică securitatea și forțează clientul să cunoască rețeaua internă (porturile și adresele IP ale fiecărui serviciu).

Pentru a rezolva aceste probleme, sistemul implementează șablonul **API Gateway**. Componenta **api-gateway** a fost dezvoltată ca un proiect Spring Boot separat și acționează ca un punct unic de intrare (*Single Point of Entry*) pentru toate cererile HTTP venite dinspre frontend.

1.2.1 Tehnologii și Dependențe Utilizate

Proiectul a fost configurat utilizând **Java 23** și **Spring Boot 3.4.1**. Funcționalitatea centrală de rutare este asigurată de dependența **spring-cloud-starter-gateway** (din suita Spring Cloud 2024.0.0), inclusă în fișierul **pom.xml**. Această tehnologie folosește un model non-blocant (bazat pe Spring WebFlux și Netty) pentru a ruteze eficient un volum mare de cereri concurente.

1.2.2 Configurarea Rutelor (Routing)

Logica de orchestrare și rutare a cererilor nu necesită scrierea de cod Java complex, ci este definită declarativ în fișierul **application.yml**. Serverul Gateway rulează pe portul **9000**.

Mecanismul funcționează prin definirea unor *predicate* (reguli de potrivire a URL-urilor). Când aplicația client face un apel către portul 9000, Gateway-ul interceptează cererea, analizează calea (**Path**) și o redirecționează invizibil către microserviciul responsabil, pe portul său intern.

Mai jos este prezentată structura de configurare a celor 6 microservicii dezvoltate:

```
1 server:
2   port: 9000
3
4 spring:
5   cloud:
6     gateway:
7       routes:
8         # 1. Microserviciul Utilizatori (Gestionare conturi & autentificare)
9         - id: utilizatori-service
10          uri: http://localhost:8081
11          predicates:
12            - Path=/api/utilizatori/**
13
14         # 2. Microserviciul Zboruri (Gestiunea rutelor si aeroporturilor)
15         - id: zboruri-service
16          uri: http://localhost:8080
17          predicates:
18            - Path=/api/zboruri/**
19
20         # 3. Microserviciul Bilete (Procesarea vanzarilor)
21         - id: bilete-service
22          uri: http://localhost:8082
23          predicates:
24            - Path=/api/bilete/**
25
26         # 4. Microserviciul Export (Generare PDF, CSV etc.)
```

```

27     - id: export-service
28       uri: http://localhost:8083
29       predicates:
30         - Path=/api/export/**
31
32     # 5. Microserviciul Statistici (Agregare date dashboard)
33     - id: statistici-service
34       uri: http://localhost:8084
35       predicates:
36         - Path=/api/statistici/**
37
38     # 6. Microserviciul Notificari (Alerte Email, SMS, WhatsApp)
39     - id: notificari-service
40       uri: http://localhost:8085
41       predicates:
42         - Path=/api/notificari/**

```

Listing 1: Configurarea rutelor în application.yml

1.2.3 Avantajele aduse arhitecturii

Clasa principală de execuție, `ApiGatewayApplication.java`, este o clasă standard Spring Boot adnotată cu `@SpringBootApplication`, fără logică de business adițională. Delegarea rutării către `application.yml` oferă următoarele avantaje strategice:

- **Decuplare totală:** Aplicația JavaFX conține un singur *Base URL* (`http://localhost:9000`). Dacă un microserviciu este mutat pe alt port sau server, codul aplicației client nu necesită nicio modificare; actualizarea se face exclusiv în YAML-ul Gateway-ului.
- **Securitate centralizată:** Gateway-ul formează un paravan. Microserviciile din spate (porturile 8080-8085) pot fi ascunse în spatele unui firewall, expunând pe internet doar portul 9000.
- **Mentenanță simplificată:** Adăugarea unui nou microserviciu în viitor presupune adăugarea a doar 4 linii de configurare în fișierul YAML.

1.3 Harta de context (Context Map)

Harta de context delimitează clar granițele (Bounded Contexts) identificate în cadrul sistemului, mapând entitățile conceptuale pe sub-domenii distincte, care au devenit ulterior microservicii independente.

S-au identificat următoarele contexte delimitate principale:

- **Bounded Context - Zboruri:** Gestionează agregatul Zbor (Aggregate Root) și entitatea Aeroport.
- **Bounded Context - Bilete:** Are ca Aggregate Root entitatea Bilet, fiind responsabil de trasabilitatea vânzărilor.
- **Bounded Context - Utilizatori:** Gestionează agregatul Utilizator și permisiunile bazate pe roluri.
- **Bounded Context - Notificări:** Gestionează sistemul de alertare (Email, SMS, WhatsApp).
- **Bounded Context - Statistici & Export:** Contexte cu rol de suport și agregare a datelor (folosind design pattern-ul *Strategy* pentru exportul în diverse formate).

Diagrama evidențiază și zona de **Common types**, care conține *Value Objects* (precum ZborID și UtilizatorID) folosite pentru a asigura comunicarea slab cuplată între microservicii.

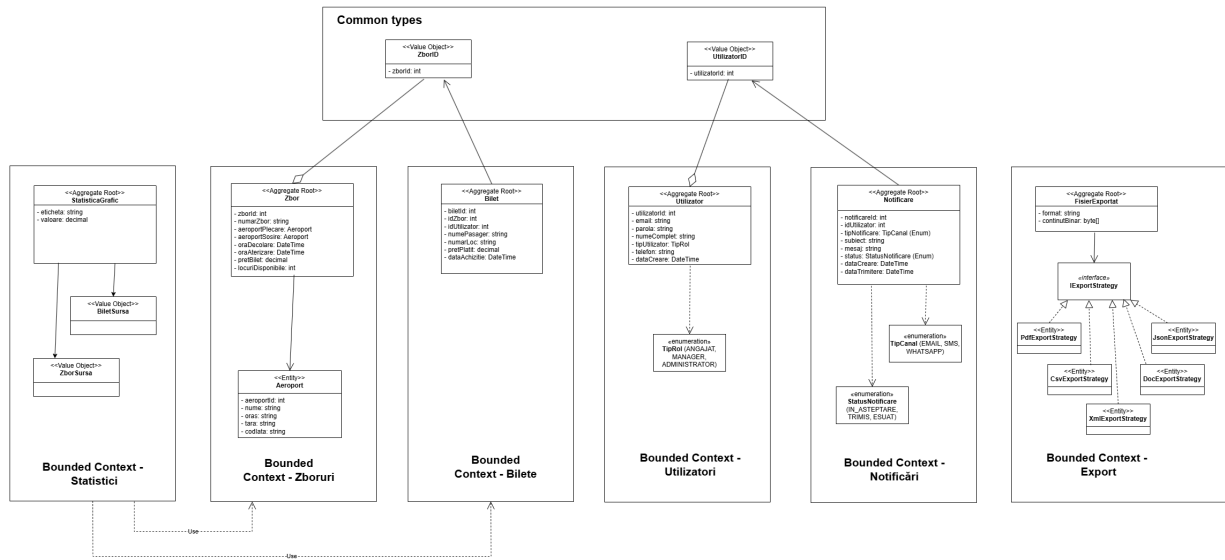


Figura 1: Harta de context a aplicației (Conform DDD)

1.4 Arhitectura internă a microserviciilor (Privire de ansamblu)

Pentru a asigura uniformitatea, s-a adoptat o structură arhitecturală de tip Clean Architecture pentru microservicii. Pachetul **Domain** conține logica de business pură și contractele (interfețele), fiind complet izolat. Pachetul **Infrastructure** implementează persistența datelor, **Services** orchestrează cazurile de utilizare, iar **Controllers** expune interfața REST.

Notă arhitecturală: Deși aceasta este structura de bază, unele microservicii prezintă ușoare variații arhitecturale în funcție de scopul lor specific (de exemplu, microserviciul de Export nu folosește un strat de DAO, ci implementează șablonul *Strategy* pentru generarea fișierelor, iar microserviciul de Statistici acționează ca un orchestrator care comunică cu alte servicii în loc să acceseze o bază de date proprie).

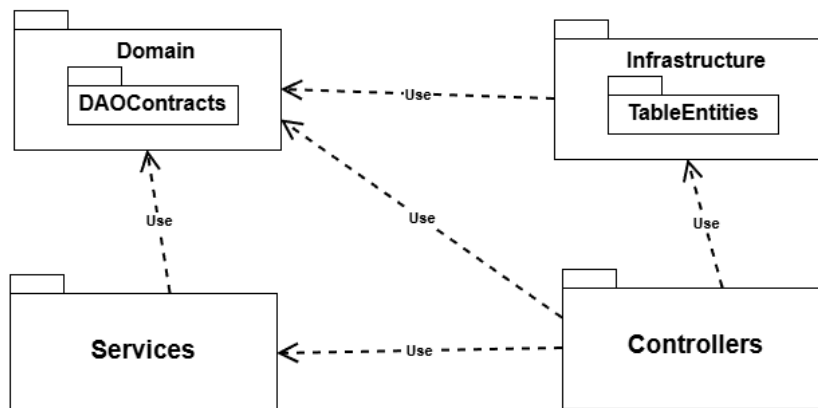


Figura 2: Diagrama de pachete generică pentru structura unui microserviciu

1.5 Arhitectura Aplicației Client (MVVM)

1.6 Diagrama de Pachete

Diagrama de pachete ilustrează organizarea logică a modulelor din cadrul aplicației client, menținând un nivel scăzut de cuplare între componentele sistemului.

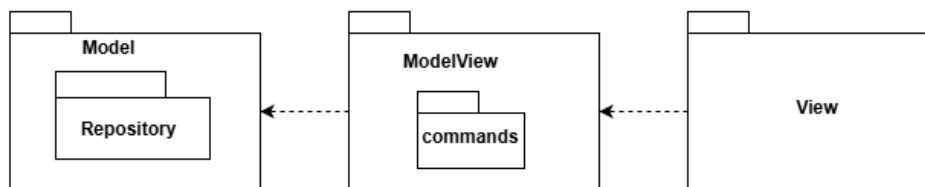


Figura 3: Diagrama de Pachete a aplicației client

Aplicația frontend este structurată în trei pachete principale, respectând paradigma MVVM: **Model**, **ModelView** (ViewModel) și **View**. Pachetul *Model* încapsulează logica datelor și interacțiunea cu API-urile externe (prin sub-pachetul *Repository*). Pachetul *ModelView* acționează ca un strat intermediar; acesta conține logica de prezentare și gestionează acțiunile utilizatorului prin intermediul sub-pachetului *commands*. Pachetul *View* este responsabil exclusiv cu randarea interfeței grafice. Dependentele sunt strict direcționale (View depinde de ModelView, iar ModelView depinde de Model), asigurând un cod ușor de testat și de întreținut.

1.7 Diagrame de Clase

Datorită complexității ridicate a sistemului și a multitudinii de funcționalități cerute de specificații, diagrama de clase completă a aplicației client a fost împărțită în mai multe diagrame specifice fiecărui tip de utilizator (rol). La nivel conceptual și arhitectural, întregul proiect principal este format din reuniunea acestor diagrame. Toate secțiunile respectă strict șablonul MVVM și utilizează design pattern-ul *Command* pentru declanșarea acțiunilor.

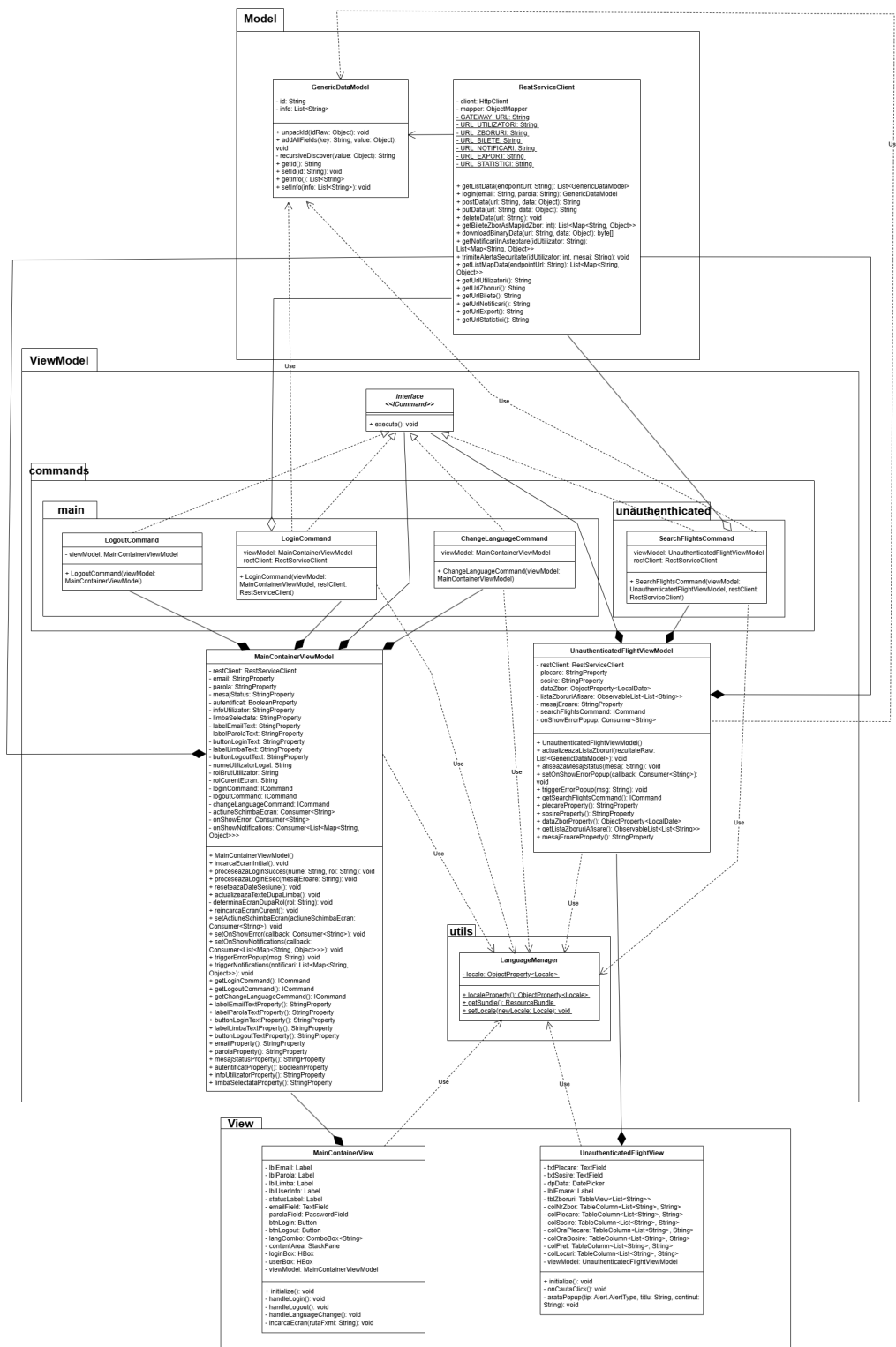


Figura 4: Diagrama de Clase - Modulul Neautentificat

Această diagramă modelează funcționalitățile de bază accesibile public, precum căutarea zborurilor și autentificarea. Clasa `MainContainerViewModel` controlează fluxul de login, iar `UnauthenticatedFlightViewModel` gestionează datele afișate în interfața de căutare a zborurilor. Extragerea datelor se face prin `RestServiceClient` aflat în *Model*, garantând izolarea interfeței grafice de apelurile de rețea.

1.7.2 Utilizator tip Angajat

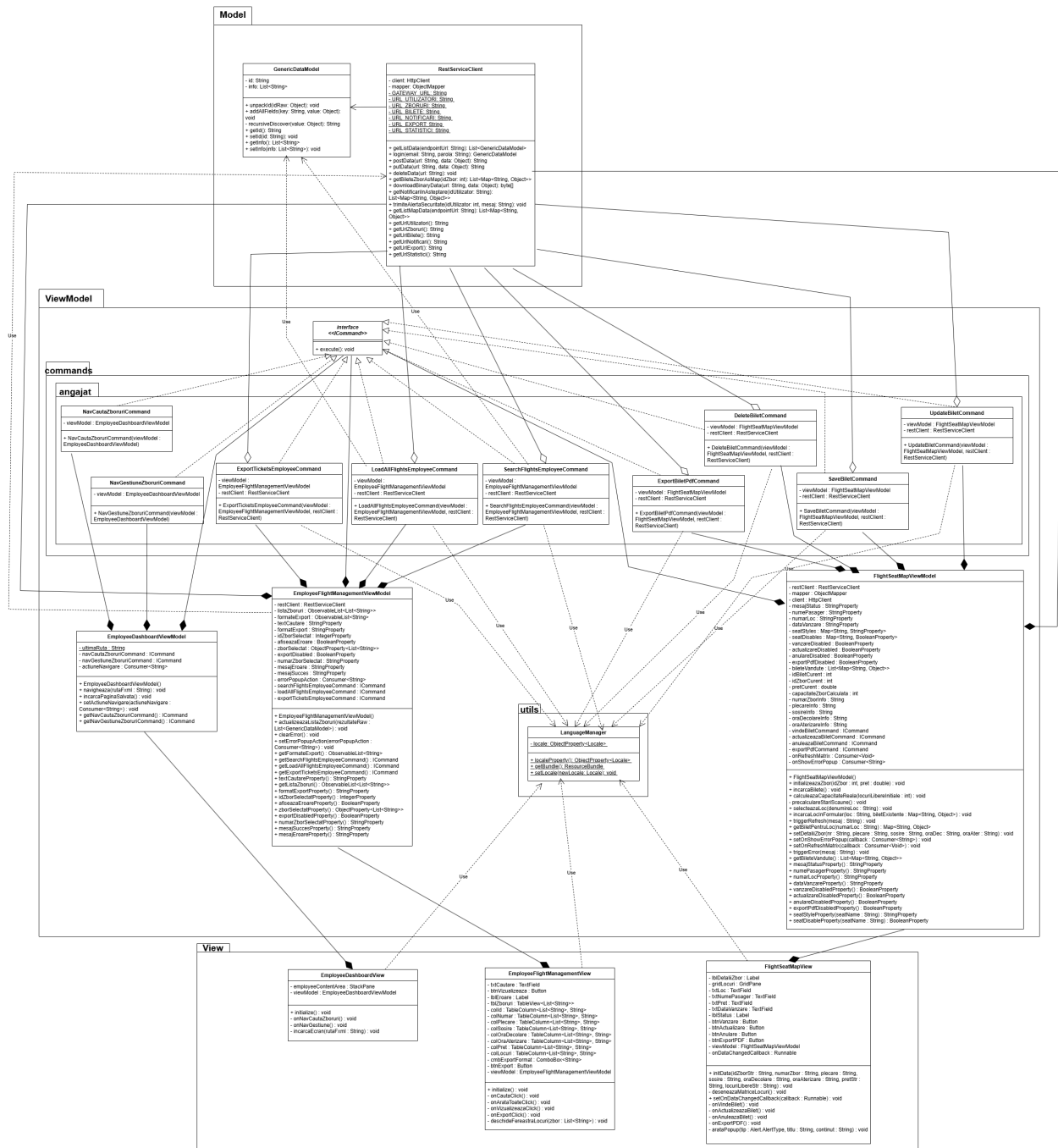


Figura 5: Diagrama de Clase - Modulul Angajat

Pentru angajat, sistemul expune clase dedicate gestionării zborurilor și procesului de vânzare a biletelor. `FlightSeatMapViewModel` joacă un rol central în reprezentarea grafică a locurilor din avion, iar funcționalitățile de export (PDF, CSV) sunt încapsulate în comenzi dedicate precum `ExportTicketsEmployeeCommand`. Totul este coordonat prin clienți REST generici care comunică cu backend-ul.

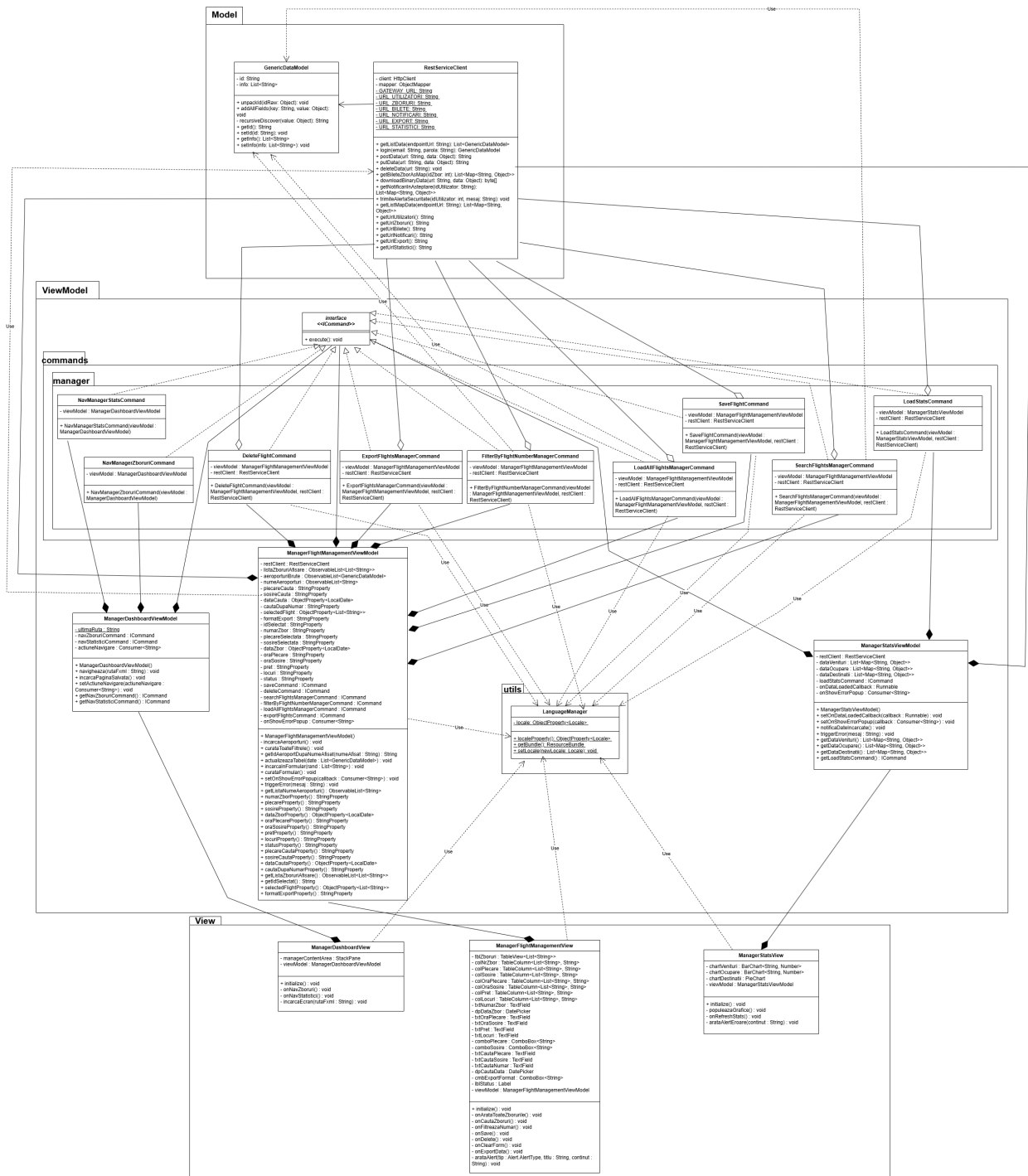


Figura 6: Diagrama de Clase - Modulul Manager

Această diagramă detaliază instrumentele de gestiune a ofertei agenției. Componentele sunt axate pe operațiile CRUD asupra zborurilor și pe vizualizarea performanței. Acțiunile managerului de a adăuga, modifica sau șterge zboruri sunt transformate în comenzi (ex. `SaveFlightCommand`, `DeleteFlightCommand`) executate de `ManagerFlightManagementViewModel`. Graficele statistice sunt alimentate de `ManagerStatsViewModel`.

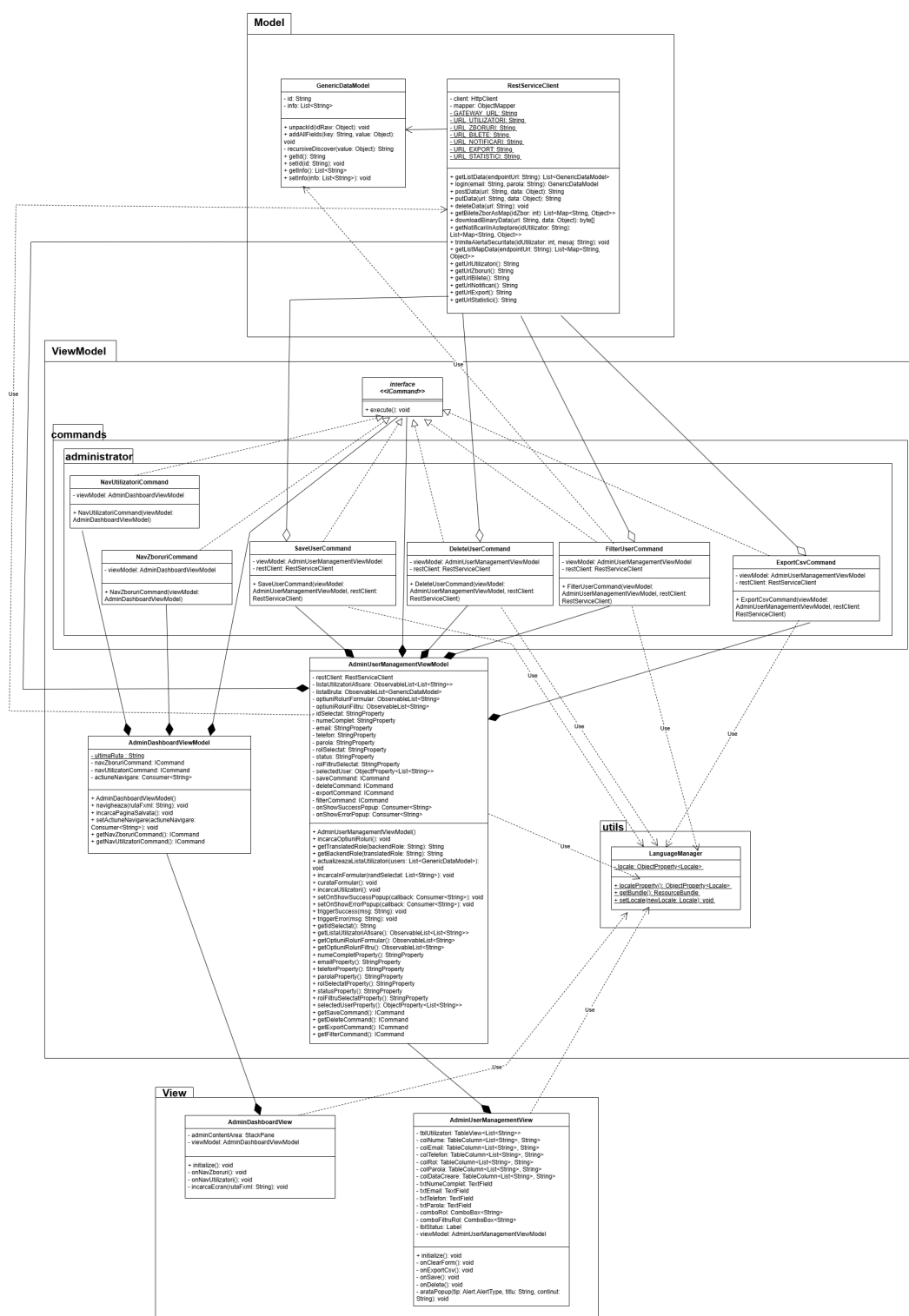


Figura 7: Diagrama de Clase - Modulul Administrator

Rolul de administrator este izolat în propriul său spațiu arhitectural, având responsabilități stricte de gestiune a conturilor angajaților și managerilor. Interfața `AdminUserManagementView` se leagă de `AdminUserManagementView` care orchestrează apelurile CRUD pentru utilizatori, delegând validările și trimiterea notificărilor către micro-serviciile aferente, prin intermediul interfetelor din *Model*.

1.8 Diagrama de Componente

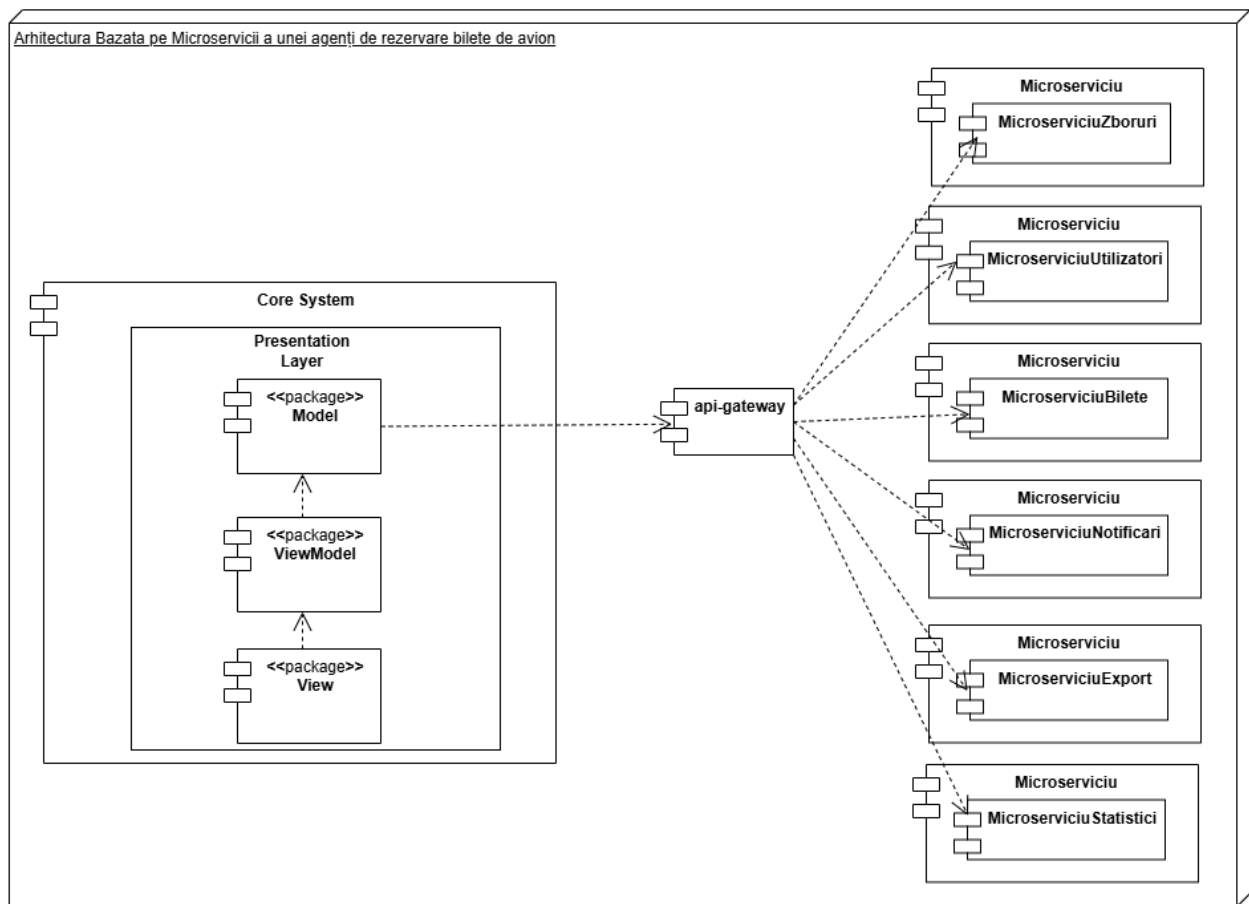


Figura 8: Diagrama de Componente

Diagrama de componente oferă o vedere de ansamblu (high-level) a întregului ecosistem. Sistemul este divizat în aplicația de tip *Core System* (frontend-ul structurat în straturile MVVM) și o arhitectură de backend bazată pe microservicii.

Fiecare microserviciu (Zboruri, Utilizatori, Bilete, Notificări, Export, Statistici) este dezvoltat separat în propriul său mediu și abstractizează o funcționalitate de business distinctă. Aplicația client comunică exclusiv prin componenta **api-gateway**, care rutează inteligent cererile către microserviciul corespunzător. Această decizie arhitecturală ascunde complexitatea backend-ului față de aplicația desktop și permite dezvoltarea, testarea și scalarea fiecărui modul în mod independent.

1.9 Diagrama de Dezvoltare (Deployment)

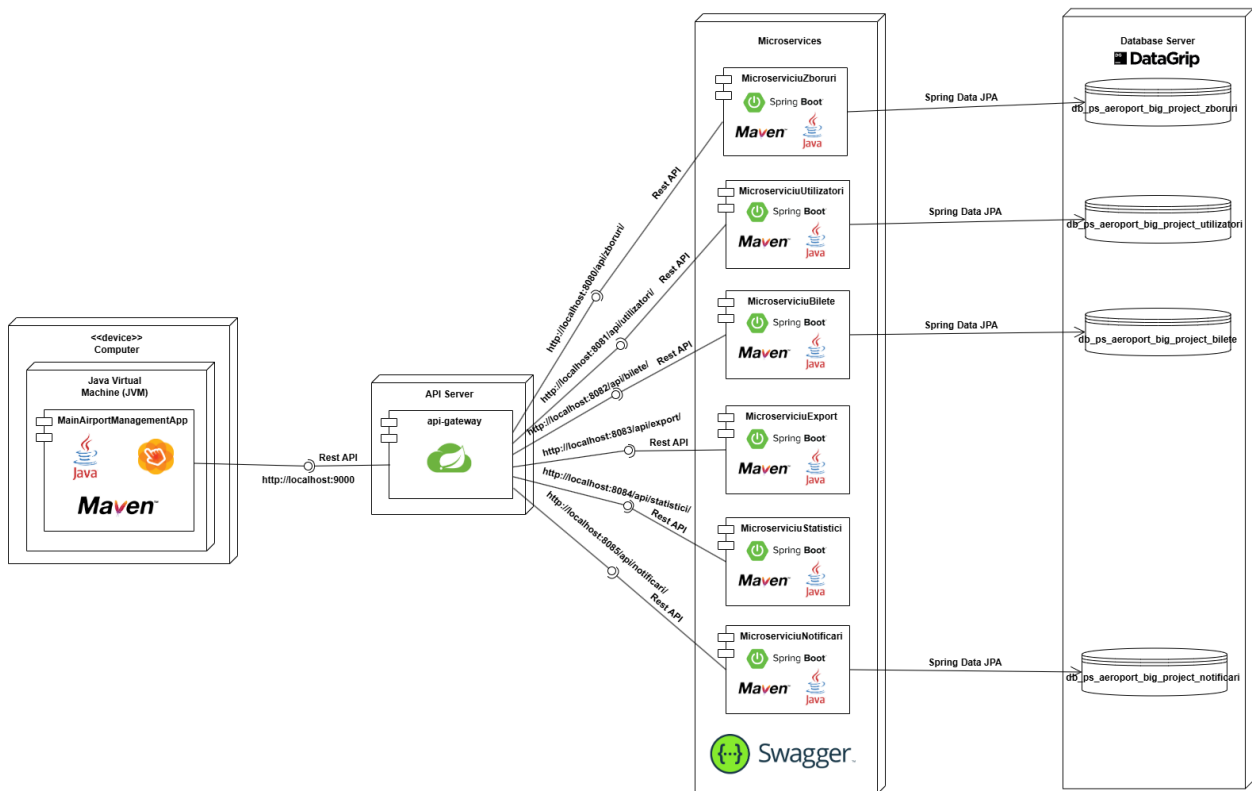


Figura 9: Diagrama de Dezvoltare

Diagrama de dezvoltare ilustrează topologia fizică și mediul de execuție al sistemului informatic. Avem trei zone logice majore:

1. **Mediul Client (Computer):** Găzduiește aplicația *MainAirportManagementApp* rulând în interiorul unui Java Virtual Machine (JVM).
2. **Mediul Server (Microservices & API Gateway):** Aplicația client accesează *API Server*-ul (pe portul 9000). Acesta redirecționează apelurile REST către microserviciile specifice. Fiecare microserviciu este o aplicație *Spring Boot* de sine stătătoare, gestionată prin Maven și rulând pe propriul său port (de la 8080 la 8085). Această arhitectură este documentată și testată integrat folosind *Swagger*.
3. **Mediul Baze de Date (Database Server):** Datele sunt persistate în baze de date distincte pentru fiecare context (zboruri, utilizatori, bilete etc.), comunicarea dintre microservicii și baze realizându-se standardizat prin intermediul *Spring Data JPA*.